Laboratorio 4:

Unidad Curricular: Tecnologías de Microprocesamiento
Universidad Tecnológica (UTEC) - Ingeniería Mecatrónica

Tizziano Travers (C.I.: 5.623.448-1)
Bruno Bentancourt (C.I.: 5.495.926-3)
Giuliano Gilardoni (C.I.: 5.644.255-9)
Docentes: Jesús Martínez / Marcelo Díaz

I. Introducción

En este cuarto laboratorio se trabaja con el microcontrolador ATmega328P para resolver diferentes problemas prácticos de ingeniería mecatrónica. El objetivo principal es programar en lenguaje C (AVR) aprovechando los recursos internos del chip, como los temporizadores, las entradas analógicas, la memoria EEPROM y los módulos de comunicación.

El laboratorio se compone de cinco proyectos prácticos: la emulación de un cajero automático con tarjeta RFID y almacenamiento en EEPROM; un simulador de proceso industrial distribuido en dos microcontroladores mediante comunicación SPI; un sistema de alarma supervisado de forma inalámbrica por Bluetooth; un juego de memoria sobre una matriz LED RGB 8×8 controlado por joystick; y un medidor de inclinación utilizando el sensor MPU6050 vía I2C.

Para organizar la lógica de programa y asegurar que el sistema responda a tiempo ante cualquier evento o sensor, cada ejercicio fue desarrollado utilizando Máquinas de Estados Finitos (FSM).

II. Objetivos

II-A. Objetivo General

Programar, simular y probar diferentes sistemas embebidos utilizando el microcontrolador ATmega328P, integrando sensores, actuadores, memoria no volátil EEPROM y comunicación entre dispositivos a través de Máquinas de Estados Finitos.

II-B. Objetivos Específicos

- Utilizar los protocolos de comunicación SPI, I2C y Bluetooth/UART para enviar y recibir datos entre el microcontrolador, pantallas y sensores.
- Organizar la lógica de cada programa utilizando Máquinas de Estados Finitos (FSM) para lograr un código ordenado y eficiente.
- Guardar información importante en la memoria EEPROM (como saldos de cuentas, claves PIN y récords) para evitar que se pierda al apagar el circuito.
- Configurar las entradas analógicas (ADC), las salidas de pulso (PWM) y las interrupciones del ATmega328P para controlar motores, servomotores, botones y matrices LED.
- Validar el funcionamiento de cada ejercicio mediante simulación en Proteus y montaje práctico en protoboard.

III. Materiales y Componentes Requeridos

IV. Listado General de Materiales y Componentes

A continuación se describen los componentes principales utilizados en el desarrollo de los laboratorios, incluyendo su función dentro de los sistemas embebidos y el espacio reservado para su ilustración gráfica.

IV-A. Microcontroladores y Procesamiento

IV-A1. Microcontrolador ATmega328P (Arduino UNO / Nano): Microcontrolador de 8 bits basado en arquitectura AVR RISC. Encargado del procesamiento principal de código, gestión de periféricos (SPI, TWI/I2C, ADC, Timers/PWM) y ejecución de las máquinas de estados.



Figura 1. Microcontrolador ATmega328P.

IV-B. Sensórica y Lectura

IV-B1. Lector RFID RC522 y Tarjetas MIFARE: Módulo de lectura y escritura por radiofrecuencia (13.56 MHz) mediante interfaz SPI. Se utiliza para la identificación no inductiva de usuarios a través del UID de 4 bytes almacenado en las tarjetas MIFARE.



Figura 2. Módulo Lector RFID RC522 y Tarjeta MIFARE.

IV-B2. Sensor de Luz LDR (Fotoresistencia): Sensor analógico cuya resistencia varía en función de la intensidad de luz incidente. Se utiliza para la detección de presencia de objetos en la banda transportadora y apertura de accesos.



Figura 3. Sensor óptico LDR (Fotoresistencia).

IV-B3. Sensor de Temperatura y Humedad DHT11: Sensor digital que transmite mediciones térmicas mediante un protocolo propietario de un solo hilo (One-Wire). Permite monitorear la temperatura para disparar estados de parada de emergencia.

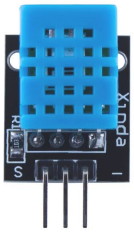


Figura 4. Sensor de temperatura y humedad DHT11.

IV-B4. Sensor IMU MPU6050 (Acelerómetro y Giroscopio): Unidad de medición inercial de 6 grados de libertad (3 ejes de aceleración y 3 de velocidad angular) con interfaz I2C. Utilizada para calcular ángulos de inclinación (Pitch y Roll) en tiempo real.



Figura 5. Sensor IMU MPU6050 (Acelerómetro y Giroscopio).

IV-C. Interfaces de Entrada y Control

IV-C1. Teclado Matricial 4x4: Arreglo de 16 pulsadores organizados en 4 filas y 4 columnas. Permite el ingreso numérico de claves PIN, montos monetarios y selección de menús mediante técnicas de barrido digital.



Figura 6. Teclado matricial de 4×4.

IV-C2. Módulo Joystick Analógico KY-023: Dispositivo de entrada compuesto por dos potenciómetros (ejes X e Y) y un pulsador digital integrado (SW), utilizado para el control bidimensional de cursores en pantalla.



Figura 7. Módulo Joystick analógico KY-023.

IV-C3. Potenciómetro: Resistencia variable manual configurada como divisor de tensión analógico, utilizada para ajustar parámetros del sistema en tiempo real, como la velocidad de la banda transportadora.



Figura 8. Potenciómetro analógico de ajuste.

IV-D. Visualización, Actuadores y Comunicación

IV-D1. Pantalla LCD 16x2 con Módulo I2C (PCF8574): Pantalla alfanumérica de 2 líneas por 16 caracteres. El adaptador I2C reduce la cantidad de pines necesarios a solo 2 líneas de comunicación de datos (SDA/SCL).



Figura 9. Pantalla LCD 16x2 con adaptador I2C PCF8574.

IV-D2. Matriz LED RGB 8x8 (WS2812B / MAX7219): Arreglo de 64 diodos LED direccionables utilizado para la representación gráfica de secuencias luminosas, visualización de juegos y simulador de nivelación de burbuja.



Figura 10. Matriz LED RGB de 8×8.

IV-D3. Motor DC con Driver L293D / L298N: Motor de corriente continua acoplado a un controlador Puente H (L293D), encargado de la tracción y modulación de velocidad por PWM de la banda transportadora.

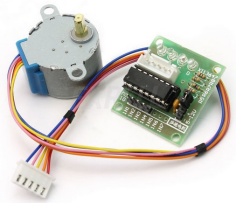


Figura 11. Motor DC con integrado Driver L293D.

IV-D4. Servomotor SG90: Actuador posicional controlado por modulación de ancho de pulso (PWM a 50 Hz). Permite un control angular preciso ($0° - 180°$) para la función de pistón desviador de piezas.



Figura 12. Servomotor SG90.

IV-D5. Módulo Bluetooth HC-05: Transceptor de comunicación inalámbrica serie (UART) que establece un enlace bidireccional transparente entre el microcontrolador y un dispositivo remoto (celular).



Figura 13. Módulo de comunicación Bluetooth HC-05.

IV-D6. Buzzer Activo de 5V: Transductor piezoeléctrico que emite tonos sonoros al recibir un nivel lógico alto, utilizado para señalizar alarmas de seguridad, confirmación de teclas o alertas de error.



Figura 14. Buzzer activo de 5V.

V. Problema A: Cajero Automático con Microcontrolador ATmega328P, Lector RFID RC522 y Tarjetas MIFARE

V-A. Objetivo

Diseñar e implementar un sistema embebido de cajero automático (ATM) sobre el microcontrolador ATmega328P. El sistema autentica usuarios mediante lectura de tarjetas RFID MIFARE (lector RC522) y verificación de clave PIN por teclado matricial 4×4, despliega el estado de las operaciones en una pantalla LCD 16×2 vía I2C y gestiona depósitos, retiros y consultas de saldo con almacenamiento persistente en la memoria EEPROM interna.

V-B. Metodología

El desarrollo del cajero automático se estructuró en cuatro etapas fundamentales:

1. **Identificación y Lectura RFID (Bus SPI):** Interfaz hardware SPI con el módulo RC522 para extraer el identificador único (UID de 4 bytes) de las tarjetas pasivas MIFARE.
2. **Interfaz de Usuario y Autenticación (GPIO y TWI/I2C):** Barrido continuo de un teclado matricial 4×4 por filas y columnas para el ingreso de clave PIN y montos de dinero. La visualización de instrucciones y saldos se realiza mediante pantalla LCD 16×2 controlada con expansor PCF8574 (100 kHz).
3. **Persistencia de Datos en EEPROM:** Almacenamiento no volátil para 3 cuentas de usuario en la memoria EEPROM del microcontrolador. Se incluye una firma inicial (*Magic Key* `0x44`) que determina si la memoria ya contiene datos guardados o si debe precargarse con los valores por defecto.
4. **Señales de Realimentación y Alarma:** Un LED verde indica validaciones y transacciones exitosas, un LED rojo señala errores en la clave o lectura, y un buzzer activo emite ráfagas sonoras de advertencia ante 3 intentos fallidos consecutivos de PIN.

V-C. Máquina de Estados

El control central del cajero se organizó mediante una Máquina de Estados Finitos (FSM) de 7 estados definidos, tal como se detalla a continuación:



Figura 15. Diagrama de la Máquina de Estados Finitos (FSM) para el Cajero Automático.

El flujo operativo mostrado en la Fig. 15 funciona de la siguiente manera:

- **S0 (INICIALIZACION):** Configura los buses TWI (I2C) y SPI, inicializa el sensor RC522 y verifica el byte mágico `0x44` en la posición `0x0000` de la EEPROM. Si coincide, carga las 3 cuentas existentes a RAM; si no, escribe las cuentas iniciales por defecto.
- **S1 (ESPERA_TARJETA):** Mantiene los actuadores apagados y muestra en LCD `."Acerque Tarjeta"`. Al aproximar un tag, lee su UID: si no está registrado en la EEPROM, emite señal de error en LED rojo y muestra el UID desconocido; si es válido, resetea los intentos de PIN y salta a **S2**.
- **S2 (INGRESO_PIN):** Solicita la clave de 4 dígitos ocultando los caracteres con asteriscos (`****`). Si la clave coincide con la guardada en la EEPROM, emite un tono de confirmación y pasa a **S3**. Si se equivoca 3 veces consecutivas, activa la alarma sonora continua de seguridad y fuerza el cierre de sesión a **S1**. También permite cancelar presionando la tecla `*`.
- **S3 (MENU_PRINCIPAL):** Despliega las opciones en pantalla: `1:Sal 2:Ret 3:Dep / *:Salir`. El usuario presiona la tecla correspondiente para acceder a los submenús o salir.
- **S4 (CONSULTA_SALDO):** Muestra en pantalla el saldo disponible de la cuenta en formato monetario durante 3 segundos y retorna automáticamente al menú principal.
- **S5 (RETIRO_MONTO):** Solicita el monto a extraer por teclado confirmando con `#`. Comprueba si el saldo actual es suficiente: si alcanza, lo descuenta y actualiza inmediatamente la EEPROM; de lo contrario, alerta saldo insuficiente.
- **S6 (DEPOSITO_MONTO):** Recibe por teclado la cifra a ingresar. Suma el monto al saldo disponible de la cuenta activa y sobrescribe el valor guardado en la EEPROM.

V-D. Implementación Técnica y Memoria EEPROM

Para organizar la información de las cuentas se definieron arreglos estructurados en C de la siguiente manera:

```
typedef struct {
    uint8_t uid[4];
    char pin[5];
    uint32_t saldo;
} Cuenta;
```

Cada estructura toma un total de 13 bytes en memoria (4 bytes para el UID, 5 bytes para el PIN con terminador nulo y 4 bytes para el saldo entero de 32 bits).

V-D1. Gestión de Memoria EEPROM: Para evitar el desgaste de las celdas de memoria de la EEPROM por escrituras innecesarias en cada reinicio, la función `guardar_cuenta_EEPROM()` utiliza la rutina optimizada `eeprom_update_block()`, la cual únicamente graba en el hardware los bytes que realmente cambiaron su valor durante la transacción.

Cuadro I
MAPEO DE PINES DEL ATMEGA328P EN EL CAJERO AUTOMÁTICO

Pin ATmega328P	Etiqueta	Función / Periférico
PB1	RC522_RST	Salida Reset Lector RFID
PB2	RC522_SS	Salida Chip Select SPI (SS)
PB3	SPI_MOSI	Salida Datos SPI Master Output
PB4	SPI_MISO	Entrada Datos SPI Master Input
PB5	SPI_SCK	Salida Reloj SPI
PC1	LED_VERDE	Salida Indicador Operación Exitosa
PC2	LED_ROJO	Salida Indicador Error / Fallo
PC3	BUZZER	Salida Alarma Sonora / Tono
PC4	SDA	Bus I2C TWI (PCF8574 – LCD 16 × 2)
PC5	SCL	Bus I2C TWI (PCF8574 – LCD 16 × 2)
PD2–PD5	FILAS_KEYPAD	Salidas Barrido Teclado (4 × 4)
PD6, PD7, PB0, PC0	COLS_KEYPAD	Entradas Lectura Columnas Teclado

V-E. Explicación del Código

El código desarrollado en C AVR prescinde de librerías externas de Arduino y se organiza mediante drivers directos a los registros internos del ATmega328P:

- **Driver TWI/I2C e Interfaz LCD:** Configura el prescaler y bitrate en los registros `TWSR` y `TWBR` a 100 kHz. Envía las instrucciones al expansor I2C PCF8574 por nibles de 4 bits mediante la función `LCD_send_nibble()`, controlando las líneas de habilitación (`Enable`) y selección de registro (`RS`).
- **Driver SPI Hardware y RFID RC522:** Inicializa el periférico SPI en modo Maestro con velocidad $F_{OSC}/16$ (1 MHz) mediante el registro `SPCR`. La función `RC522_get_uid()` ejecuta la secuencia de comando `REQA` y anticolisión (`0x93 0x20`) sobre los registros del chip RC522 para obtener los 4 bytes del UID.
- **Teclado Matricial (4 × 4):** La función `Keypad_get_key()` realiza el barrido poniendo a cero secuencialmente las salidas en los pines `PD2` a `PD5`. Lee luego el nivel lógico en las entradas con

resistencias de *pull-up* activadas (`PD6`, `PD7`, `PB0` y `PC0`).

- **Bucle Principal y Lógica FSM:** Implementado mediante una estructura `switch(estado)` en el `main()`. Procesa la entrada del teclado o RFID, efectúa las operaciones de suma/resta sobre los saldos de las cuentas en memoria RAM y ejecuta las llamadas a la EEPROM para mantener la persistencia.

V-F. Montaje Físico y Conexionado

El circuito físico fue ensamblado sobre dos protoboards interconectadas. La alimentación general de 5 V regula la tensión del microcontrolador ATmega328P, del módulo LCD I2C, del teclado y de las etapas de aviso visual y sonoro. Dado que el lector RFID RC522 opera a 3.3 V, se aseguró el acoplamiento directo de sus líneas SPI a los puertos de la placa, manteniendo los tiempos de lectura estables.

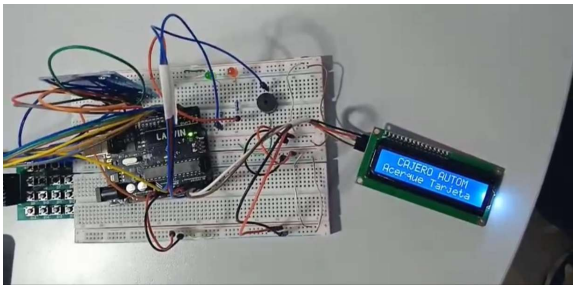


Figura 16. Vista general del prototipo físico del cajero automático con ATmega328P, lector RC522 y teclado.

El montaje físico evidencia la independencia de periféricos: mientras el bus SPI (`PB1–PB5`) trabaja la lectura del UID del usuario, el bus I2C (`PC4–PC5`) atiende en paralelo la pantalla, dejando libres los pines del puerto D para el escaneo del teclado matricial.

V-G. Simulación y Resultados

La comprobación experimental del diseño se ejecutó en Proteus 8 Professional utilizando el circuito esquemático presentado en la Fig. 17.

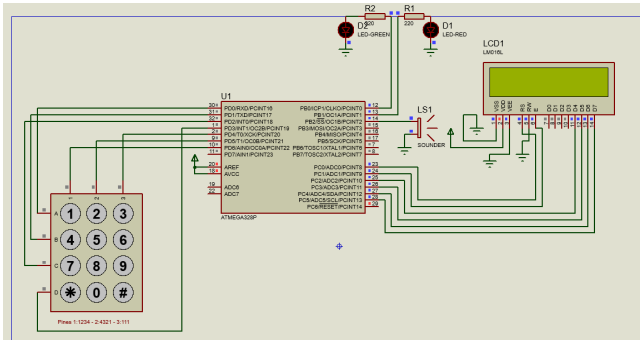


Figura 17. Circuito esquemático del cajero automático implementado en Proteus.

V-G1. Descripción del Esquemático en Proteus:

- **Microcontrolador ATmega328P** ($U1$)**:** Configurado a 16 MHz para procesar el código compilado en C AVR.
- **Teclado Matricial** 3×4**:** Conectado a los pines del Puerto D (`PD0–PD6`) para el ingreso de las teclas de emulación de tarjeta, PIN y cifras de dinero.
- **Pantalla LCD 16x2** ($LCD1$)**:** Conectada en paralelo a los pines `PC0–PC5` en modo de 4 bits para desplegar las pantallas de bienvenida, ingreso de datos, menús y montos.
- **Indicadores** ($D1, D2, LS1$)**:** El LED verde ($D2$) en `PB0` indica operaciones exitosas; el LED rojo ($D1$) en `PB1` señala fallos o errores de clave; el zumbador ($LS1$) en `PB2` emite las alertas sonoras.

V-G2. Pruebas y Escenarios Evaluados: Durante la simulación en Proteus se comprobaron experimentalmente los siguientes casos de uso del sistema:

1. **Emulación de Selección de Cuenta (RFID):** Al iniciar el sistema, la pantalla LCD muestra el mensaje inicial de bienvenida. Dado que Proteus no cuenta con el componente interactivo RC522, al presionar la tecla `'1'` en el teclado matricial se emula la lectura directa del UID asignado a la Cuenta 1.

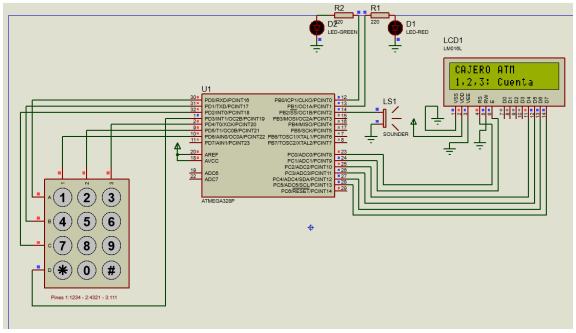


Figura 18. Estado inicial de la simulación en pantalla LCD: Espera de selección de cuenta.

2. **Autenticación de PIN:** Al seleccionar la cuenta, el sistema solicita la clave de 4 dígitos ocultándola mediante asteriscos (`PIN: ****`).

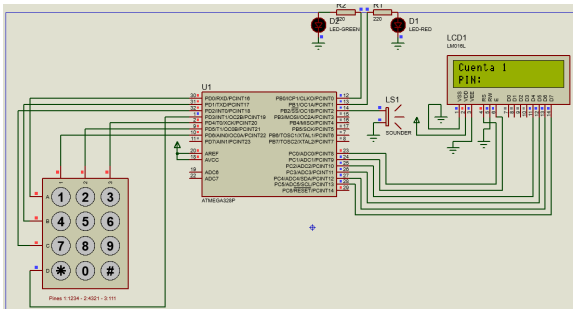


Figura 19. Ingreso del PIN de seguridad enmascarado (*).

3. **Acceso al Menú Principal:** Al ingresar la clave correcta (`"1234"`), el sistema habilita el acceso, enciende el LED verde ($D2$), emite un tono corto de confirmación

en el buzzer $LS1$ y despliega el menú principal con las opciones disponibles.

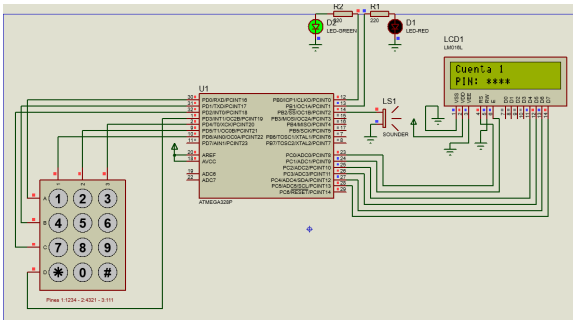


Figura 20. Acceso correcto: Encendido del LED verde y despliegue del Menú Principal.

4. **Consulta de Saldo:** Seleccionando la opción `'1'` desde el menú principal, el microcontrolador recupera el saldo de la EEPROM para la cuenta activa y lo muestra en pantalla durante 2.5 s antes de retornar al menú.

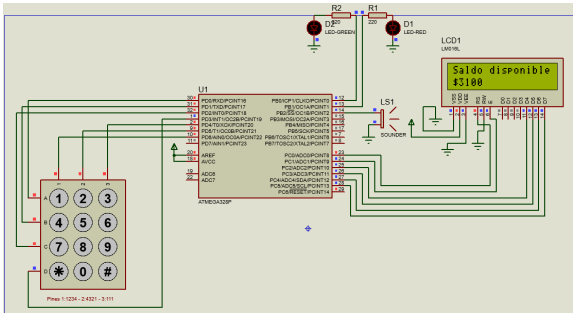


Figura 21. Despliegue en pantalla del saldo disponible leído desde la memoria EEPROM.

5. **Operaciones de Retiro y Depósito:** Al presionar las opciones `'2'` (Retiro) o `'3'` (Depósito), el sistema solicita el monto por teclado confirmando con #. Se verifica la actualización del saldo en RAM, la persistencia inmediata en la EEPROM mediante `guardar_cuenta()` y el mensaje de confirmación con el destello del LED verde.

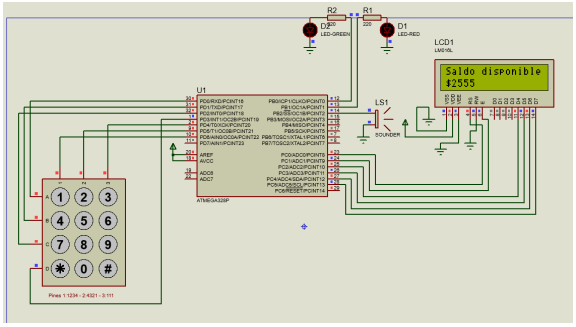


Figura 22. Confirmación de transacción realizada con éxito tras actualizar la EEPROM.

6. **Manejo de Errores y Bloqueo de Alarma:** Al ingresar una clave incorrecta, el LCD advierte `"PIN incorrecto"` y enciende el LED rojo ($D1$). Si se acumulan 3 intentos fallidos consecutivos, el LCD despliega `"3 PIN incorrectos"` y activa el buzzer $LS1$ junto al LED rojo en ráfagas de alarma sonora/luminosa.

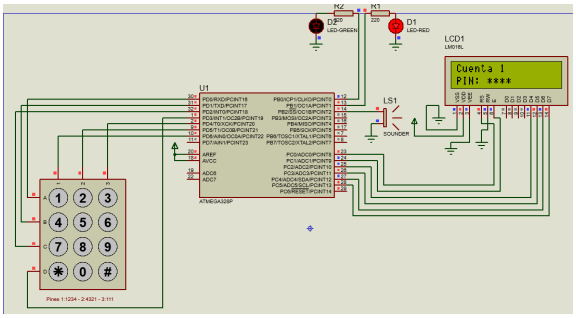


Figura 23. Respuesta ante PIN incorrecto con activación del LED rojo y señal sonora de alarma.

V-H. *Conclusiones*

La implementación del cajero automático sobre el microcontrolador ATmega328P permitió validar la integración de memoria no volátil EEPROM con periféricos E/S y una interfaz de usuario interactiva mediante una Máquina de Estados Finitos (FSM).

La emulación de la lectura RFID a través del teclado matricial en Proteus resolvió eficazmente la limitación de la falta de componentes nativos RC522 en la simulación, asegurando la verificación de toda la lógica de cuentas, claves PIN y transacciones financieras. El uso optimizado de la función `eeprom_update_block()` garantizó la conservación duradera de la información reduciendo el desgaste de la memoria no volátil del microcontrolador.

VI. Problema B: Simulador de Proceso Industrial con Banda Transportadora usando 2x ATmega328P en Arquitectura Maestro-Esclavo SPI

VI-A. *Objetivo*

Diseñar, programar y simular un sistema distribuido de control industrial basado en dos microcontroladores ATmega328P en arquitectura Maestro-Esclavo sobre bus SPI. El sistema emula una línea de producción automatizada compuesta por una banda transportadora de velocidad variable, un pistón desviador accionada por servomotor, detección optoelectrónica de piezas (LDR), monitoreo térmico del motor mediante sensor DHT11, visualización local en pantalla LCD 16×2 vía I2C y un protocolo de parada de emergencia con alarmas audiovisuales.

VI-B. *Metodología*

El desarrollo del simulador industrial se estructuró dividiendo las tareas de control y potencia entre ambos nodos para optimizar los tiempos de ejecución en tiempo real:

1. **Nodo Maestro (U1 – Toma de Decisiones e Interfaz):** Encargado de la lectura analógica/digital de los sensores del proceso (sensor LDR para piezas, potenciómetro

para velocidad, sensor digital DHT11 para temperatura y pulsador de marcha/paro). Gestiona la interfaz gráfica mediante una pantalla LCD 16×2 conectada por el bus I2C/TWI (100 kHz) y transmite los comandos de control hacia el nodo esclavo a través del bus SPI.

2. **Nodo Esclavo (U4 – Control de Actuadores):** Opera dedicado de forma exclusiva a la ejecución sobre la etapa de potencia. Recibe las instrucciones SPI por interrupción, controla el motor de la banda transportadora (paso a paso o DC mediante driver L293D), genera la señal PWM por hardware (Timer1) para el servomotor del pistón y maneja los indicadores lógicos (LED verde de marcha, LED rojo de falla) y la alarma sonora (buzzer activo).
3. **Protocolo de Comunicación SPI Robusto:** Implementación de un esquema de transferencia sincrónica mediante tramas cortas de 2 bytes (`[COMANDO][VALOR]`), con resincronización de trama por hardware utilizando la línea *Slave Select* (SS) e interrupciones de cambio de pin (PCINT).

VI-C. Arquitectura SPI y Trama de Comunicación

La comunicación entre microcontroladores se realiza en modo Maestro con preescalador de reloj $F_{OSC}/16$ (1 MHz). El Maestro gobierna la línea SS (pin `PB2`) bajándola a nivel lógico bajo antes de iniciar la transmisión y elevándola al finalizar.

Las órdenes se empaquetan en tramas fijas de 2 bytes:

$$\text{Trama SPI} = [\text{BYTE_0: COMANDO}]\ [\text{BYTE_1: VALOR}]$$

Los comandos parametrizados en el protocolo de la aplicación son:

- `CMD_STOP (0x00)`: Ordena la detención inmediata de la banda transportadora y apaga las salidas de indicación.
- `CMD_START (0x01)`: Activa la marcha normal de la línea de producción y enciende el LED verde.
- `CMD_SPEED (0x02)`: Transmite el valor del retardo entre pasos en milisegundos (derivado de la lectura del potenciómetro) para ajustar dinámicamente la velocidad de la banda.
- `CMD_PISTON (0x03)`: Notifica la presencia de una pieza detectada por el sensor LDR, desencadenando la secuencia de expulsión por el servomotor.
- `CMD_EMERGENCY (0x04)`: Dispara el estado de falla crítica por sobrecalentamiento ($> 40\,°$C), bloqueando los motores y activando la alarma audiovisual intermitente a 5 Hz.

VI-D. Máquina de Estados

El comportamiento global del simulador industrial está regido por una Máquina de Estados Finitos (FSM) coordinada entre ambos nodos, cuyo flujo principal se ilustra en la Fig. 24.

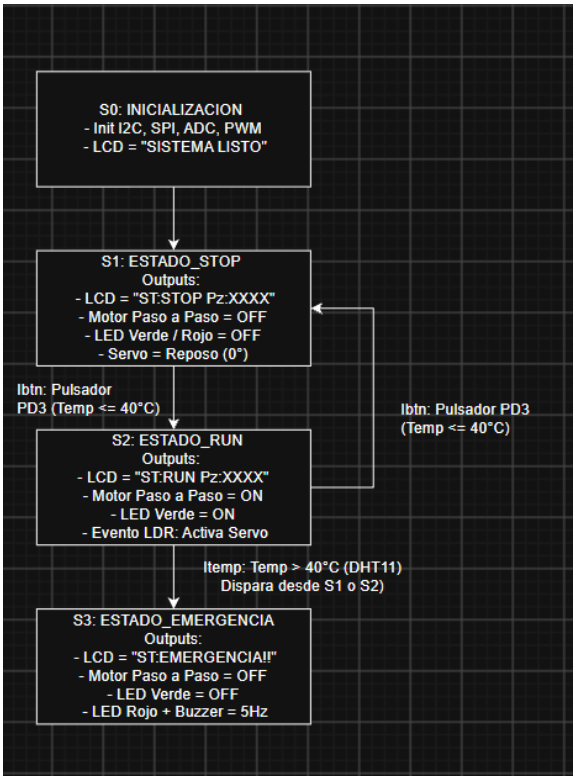


Figura 24. Diagrama de la Máquina de Estados Finitos (FSM) para el Simulador Industrial Maestro-Esclavo.

Las funciones principales de cada estado definido son:

- **S0 (INICIALIZACION):** Configura los registros de periféricos (SPI en ambos nodos, I2C/TWI, ADC de 10 bits, Timer1 en Fast PWM). Muestra en la pantalla LCD el mensaje inicial `"SISTEMA LISTO"` y posiciona el servomotor en su ángulo de reposo (0°).
- **S1 (ESTADO_STOP):** Mantiene la banda transportadora apagada, el servomotor en reposo y los indicadores apagados. La pantalla LCD despliega `"ST:STOP Pz:XXXX"`. Al presionar el pulsador de marcha conectado a `PD3` (con temperatura $\leq 40\,°$C), se envía `CMD_START` por SPI y se pasa al estado **S2**.
- **S2 (ESTADO_RUN):** Estado de producción activa. El motor de la banda gira a la velocidad ajustada por el potenciómetro en `PC1` (enviando `CMD_SPEED`) y el LED verde permanece encendido. Si el sensor LDR en `PC0` detecta el paso de una pieza ($V_{ADC} < 500$), se incrementa el contador de piezas en el LCD y se envía `CMD_PISTON` para accionar el servo a 90° durante 800 ms. Un nuevo pulso en `PD3` retorna a **S1**. Si la temperatura supera $40\,°$C, se fuerza la transición a **S3**.
- **S3 (ESTADO_EMERGENCIA):** Se activa automáticamente si el sensor DHT11 reporta una temperatura superior a $40\,°$C en los estados **S1** o **S2**. El Maestro transmite inmediatamente `CMD_EMERGENCY`, la pantalla LCD alerta `"ST:EMERGENCIA!!"`, el motor se detiene por completo y el Esclavo conmuta el LED rojo junto con el buzzer de forma intermitente a 5 Hz.

VI-E. *Implementación Técnica y Asignación de Pines*

Para garantizar un control preciso sobre la arquitectura AVR del ATmega328P sin depender de código bloqueante, la ejecución del Esclavo utiliza interrupciones por hardware para la recepción de SPI y para el control del servo por PWM.

VI-E1. Configuración del Servomotor (Timer1 Fast PWM): El control del servomotor SG90 en el nodo Esclavo se realiza mediante el Timer1 en modo Fast PWM con `ICR1` como TOP (WGM13:10 = 1110_2) y un preescalador de 8, fijando una frecuencia exacta de 50 Hz (período de 20 ms):

$$f_{PWM} = \frac{F_{CPU}}{N \cdot (1 + \text{ICR1})} = \frac{16\ \text{MHz}}{8 \cdot (1 + 39999)} = 50\ \text{Hz}$$

El ancho de pulso en el canal `OC1A` (`PB1`) se modula modificando el registro `OCR1A`: 2000 unidades representan 1.0 ms (0° en reposo) y 3800 unidades corresponden a 1.9 ms (90° de accionamiento del pistón).

Cuadro II
MAPEO DE PINES Y DISTRIBUCIÓN DE HARDWARE ENTRE MAESTRO Y ESCLAVO

Nodo	Pin ATmega328P	Etiqueta	Función / Componente Conectado
Maestro (U1)	PC0 (ADC0)	SENSOR_LDR	Lectura Analógica Sensor de Pieza (LDR)
Maestro (U1)	PC1 (ADC1)	POT_VEL	Lectura Analógica Potenciómetro de Velocidad
Maestro (U1)	PD2	DATA_DHT11	Entrada Digital Sensor de Temperatura DHT11
Maestro (U1)	PD3	BTN_START_STOP	Entrada Pulsador Marcha/Parada (Pull-Up)
Maestro (U1)	PC4, PC5	SDA, SCL	Bus I2C TWI (PCF8574 – Pantalla LCD 16 × 2)
Maestro (U1)	PB2, PB3, PB5	SS, MOSI, SCK	Salidas Bus SPI (Maestro Transmisor)
Esclavo (U4)	PB2, PB3, PB5	SS, MOSI, SCK	Entradas Bus SPI (Esclavo Receptor)
Esclavo (U4)	PD2	LED_VERDE	Salida Indicador de Estado Marcha
Esclavo (U4)	PD3	LED_ROJO	Salida Indicador de Error / Emergencia
Esclavo (U4)	PB0	BUZZER	Salida Alarma Sonora Audible
Esclavo (U4)	PB1 (OC1A)	SERVO_PISTON	Salida PWM Timer1 (Servomotor Pistón SG90)
Esclavo (U4)	PD4–PD7	MOTOR_IN1--IN4	Salidas Driver L293D (Banda Transportadora)

VI-F. *Explicación del Código*

El desarrollo del firmware en lenguaje C C-AVR para ambos microcontroladores utiliza manipulación directa de registros para maximizar la velocidad de respuesta:

- **Controlador I2C/LCD (Maestro):** Inicializa el módulo TWI a 100 kHz mediante los registros `TWBR = 72` y `TWSR = 0`. Envía instrucciones por nibles de 4 bits mediante `LCD_SendNibble()`, refrescando el conteo de piezas, la velocidad calculada (%) y la temperatura medida sin retardos bloqueantes.
- **Procesamiento ADC y DHT11 (Maestro):** Configura el multiplexor `ADMUX` con referencia AVCC y lee secuencialmente el canal 0 (LDR) y canal 1 (potenciómetro). Una rutina de lectura por software muestrea periódicamente el sensor DHT11 en el pin `PD2` extrayendo la lectura de temperatura en grados Celsius.
- **Recepción SPI por Interrupción (Esclavo):** La rutina de interrupción `ISR(SPI_STC_vect)` procesa los bytes entrantes del bus SPI. Al recibir el primer byte se almacena el comando (`rx_cmd`), y con el segundo byte se actualiza el parámetro asociado (`rx_val`), activando banderas como `trigger_piston` o modificando la velocidad `step_delay_ms`. Una interrupción por cambio de pin (`ISR(PCINT0_vect)`) sobre la línea SS resincroniza el índice del buffer en caso de ruido en el bus.
- **Bucle Principal de Actuación (Esclavo):** El `main()` del Esclavo ejecuta una máquina de estados secuencial en bucle infinito: en estado `CMD_START` conmuta secuencialmente la máscara de pasos en las salidas `PD4–PD7` para mover la banda transportadora; si la bandera de pistón está activa, ejecuta la rutina `ActuatePiston()` modificando el registro `OCR1A` del Timer1; en estado `CMD_EMERGENCY` conmuta el LED rojo y el buzzer con un retardo de 200 ms.

VI-G. *Montaje Físico y Conexionado*

El montaje físico integra la plataforma de procesamiento sobre protoboard dividida en dos bloques de control claros. Un bus de 4 hilos (MOSI, MISO, SCK, SS) interconecta ambos microcontroladores ATmega328P junto con una referencia común de masa (GND).

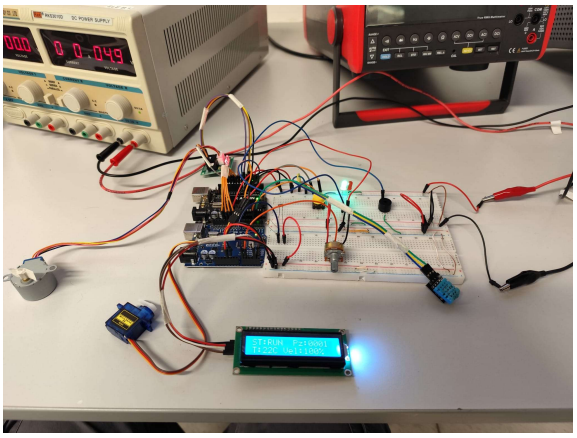


Figura 25. Vista general del prototipo físico del simulador industrial mostrando el Maestro (LCD/Sensores) y Esclavo (Driver L293D y Actuadores).

El diseño de hardware desacopla la etapa de potencia: el driver L293D y el servomotor se alimentan desde una fuente regulada externa de 5 V, evitando caídas de tensión o ruidos inductivos sobre las líneas de lectura analógica del Maestro y garantizando la estabilidad de las comunicaciones SPI y TWI.

VI-H. *Simulación y Resultados*

La validación y verificación completa del sistema distribuido se llevó a cabo en la plataforma Proteus 8 Professional mediante el circuito esquemático presentado en la Fig. 26.

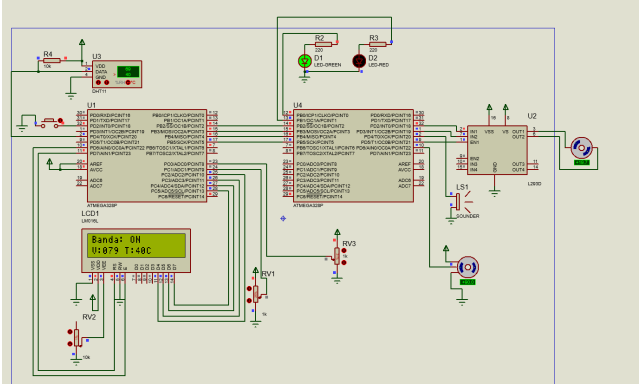


Figura 26. Circuito esquemático completo del simulador industrial Maestro-Esclavo en Proteus.

VI-I. Conclusiones

La implementación del simulador de proceso industrial sobre la arquitectura Maestro-Esclavo con dos microcontroladores ATmega328P demostró la efectividad de la distribución de carga en sistemas embebidos. El protocolo de comunicación SPI de 2 bytes ofreció una latencia mínima en la transmisión de comandos, garantizando una respuesta inmediata del servomotor y una detención de seguridad instantánea ante eventos de sobretemperatura.

VII. Problema C: Sistema de Alarma Antirrobo con Supervisión Inalámbrica usando ATmega328P y Módulo Bluetooth HC-05

VII-A. Objetivo

El objetivo principal de este proyecto es diseñar, simular e implementar un sistema distribuido de alarma antirrobo con supervisión y control inalámbrico mediante comunicación serie Bluetooth (módulo HC-05 / aplicación móvil) utilizando el microcontrolador **ATmega328P** programado en lenguaje C bare-metal.

VII-B. Metodología

El sistema opera a una frecuencia de reloj de 16 MHz e integra un sensor de movimiento discreto (botón/PIR en el pin `PD2`) y un sensor perimétrico de ventana (fotorresistencia LDR en el canal `ADC0`).

Entre los objetivos metodológicos específicos se destacan:

- Implementar una Máquina de Estados Finitos (FSM) que contemple los estados `DESARMADO`, `ARMADO` y `SILENCIADO`.
- Desarrollar dos modos de operación seleccionables: `MODO_AUTO` (activación de sirena local y aviso remoto) y `MODO_MANUAL` (notificación remota sin activación de la sirena).
- Diseñar un algoritmo de autocalibración adaptativa en el ADC para el sensor LDR, eliminando falsas alarmas provocadas por la alta variabilidad lumínica del entorno.
- Garantizar una comunicación robusta por UART a 9600 baudios inmune a la fragmentación de tramas por Bluetooth.

La metodología de desarrollo se dividió en tres fases interrelacionadas:

1. **Modelado del Hardware y Puertos GPIO:** Configuración del registro de dirección de datos (`DDRD`) y habilitación de la resistencia de *pull-up* interna en `PD2`.
2. **Diseño de Firmware en C Bare-Metal para AVR:** Desarrollo modular con funciones de inicialización de UART, conversor analógico-digital (ADC) e interrupciones por consulta en el bucle principal (*polling loop*).
3. **Depuración e Integración Inalámbrica:** Filtrado de caracteres de control de línea (`\r`, `\n`) y diseño de un protocolo de prioridades en el análisis de subcadenas con `strstr()`.

VII-C. Comunicación Bluetooth y Comandos

La comunicación serie se realiza mediante el módulo Bluetooth HC-05 conectado a los pines de transmisión serie del microcontrolador (`PD0/RXD0` y `PD1/TXD0`). La configuración del periférico USART se define para trabajar a 9600 baudios, 8 bits de datos, 1 bit de parada y sin paridad (8N1).

El cálculo del registro del generador de tasa de baudios se rige por la ecuación:

$$\text{UBRR0} = \frac{F_{\text{CPU}}}{16 \times \text{BAUD}} - 1 = \frac{16\,000\,000}{16 \times 9600} - 1 = 103 \quad (1)$$

VII-D. Máquina de Estados

El control del sistema se fundamenta en una FSM guiada por comandos recibidos vía Bluetooth y lectura continua de sensores. Los estados y sus transiciones se estructuran de la siguiente manera:

- **DESARMADO:** Todos los actuadores se encuentran inactivos (`LED_VERDE`, `LED_ROJO` y `BUZZER` apagados).
- **ARMADO:** Se enciende el `LED_VERDE` (`PD3`) e inicia el monitoreo del sensor LDR y el botón de presencia. Si se detecta intrusión en `MODO_AUTO`, conmuta a `LED_ROJO` y `BUZZER` encendidos enviando la alerta serie correspondiente.
- **SILENCIADO:** Desactiva la sirena local (`BUZZER` en `PD5`) manteniendo la vigilancia activa y el estado de alerta reportado.

VII-E. Implementación Técnica

El control de entrada/salida se realiza mediante manipulación directa de registros AVR (`PORTD`, `PIND`, `DDRD`). La recepción de datos por USART se realiza mediante lectura de la bandera `RXC0` en el registro `UCSR0A`, almacenando los caracteres en un búfer circular de 32 bytes (`bufferRX`).

VII-F. Calibración Adaptativa por Software para la LDR

El nivel de luz del laboratorio variaba significativamente respecto a un valor teórico fijo. Para evitar que la alarma se disparara de forma errónea al armar, se desarrolló una técnica de autocalibración adaptativa por software para la fotorresistencia.

En el preciso instante en que el sistema procesa la orden `ARMAR`, el conversor analógico-digital (ADC) efectúa una lectura del pin `A0` y almacena la luz ambiente del recinto en la variable global $luzBase$:

$$luzBase = \text{ADC_Read}() \quad (2)$$

Durante el estado `ARMADO`, la condición de intrusión en la zona de ventana (detencionLDR) no se compara contra un valor estático, sino contra un rango diferencial con un margen seguro ($\Delta = 200$):

$$\text{detencionLDR} = \begin{cases} 1 & \text{si valorLDR} < (luzBase - 200) \quad \text{o} \quad \text{valorLDR} \\ 0 & \text{en otro caso} \end{cases} \quad (3)$$

Esto asegura que el sistema únicamente se active si un intruso interrumpe o bloquea intencionalmente la luz sobre la fotorresistencia.

VII-G. Explicación del Código

El código fuente fue desarrollado en lenguaje C para arquitectura AVR, optimizado para ejecutarse en el microcontrolador ATmega328P operando a 16 MHz.

El programa inicia definiendo la frecuencia del cristal principal (`F_CPU = 16000000UL`) y la velocidad del puerto serie a 9600 bps (MY_UBRR = 103). Se asignan alias a los pines del Puerto D para las salidas (`LED_VERDE` en `PD3`, `LED_ROJO` en `PD4`, `BUZZER` en `PD5`) y la entrada con resistencia de *pull-up* activa (`BOTON_PIN` en `PD2`).

Para controlar el sistema se implementan dos enumeraciones (`EstadoSistema` y `ModoOperacion`), junto con el buffer `bufferRX` y la variable `luzBase`. La función `USART_Init()` configura el periférico serie en formato 8N1, mientras que `USART_SendString()` envía respuestas terminadas en `\r\n`. En `initGPIO()` y `ADC_Init()` se configuran las salidas digitales y la referencia de voltaje de 5 V con prescaler de 128 para el conversor analógico-digital (125 kHz).

La función `procesarComando()` analiza las órdenes mediante `strstr()`, priorizando `DESARMAR` para evitar falsas coincidencias con la palabra `ARMAR`. Al ejecutar `ARMAR`, se captura la lectura analógica de referencia mediante `luzBase = ADC_Read()`. Dentro del bucle principal `main()`, el sistema monitorea continuamente el buffer serie y evalúa la condición adaptativa del sensor LDR ($|\text{valorLDR} - luzBase| > 200$), utilizando las banderas `flagEventoMovimiento` y `flagEventoVentana` para evitar el reenvío redundante de tramas por Bluetooth.

VII-H. Montaje Físico y Conexionado

El circuito utiliza la placa de desarrollo basada en el ATmega328P. La asignación física de pines se especifica en la Tabla III.

Cuadro III
MAPEO DE PINES Y COMPONENTES ELECTRÓNICOS.

Pin del Microcontrolador	Componente Periférico	Función / Configuración
`PD0 (RX0)`	Módulo HC-05 (TXD)	Recepción de Comandos Serie
`PD1 (TX0)`	Módulo HC-05 (RXD)	Transmisión de Alertas Serie
`PD2`	Botón / Sensor PIR	Entrada Digital (Pull-up interna)
`PD3`	LED Verde	Salida Digital (Indicador Armado)
`PD4`	LED Rojo	Salida Digital (Indicador Alarma)
`PD5`	Buzzer Piezoeléctrico	Salida Digital (Sirena Acústica)
`ADC0 (A0)`	Fotorresistencia (LDR)	Entrada Analógica (Divisor de Tensión)

A continuación, se presentan las evidencias fotográficas del montaje físico del circuito y su validación operacional en el laboratorio:

VII-I. Simulación y Resultados

Durante las pruebas experimentales y la simulación interactiva en Proteus, se verificaron con éxito todos los escenarios operacionales del sistema de alarma antirrobo mediante la Virtual Terminal y el microcontrolador ATmega328P.

VII-I1. Validación en el Entorno de Simulación Proteus: A continuación, se describen las evidencias del funcionamiento del firmware frente a la máquina de estados finitos (FSM):

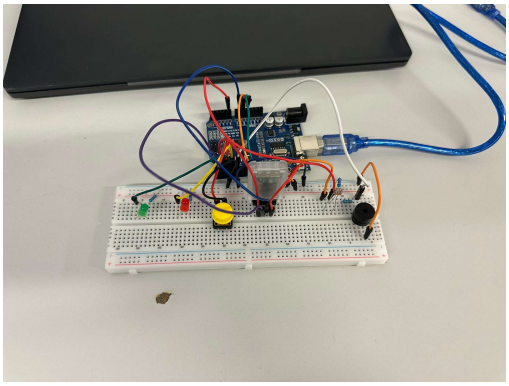


Figura 27. Montaje físico general del sistema de alarma antirrobo en protoboard acoplado a la placa ATmega328P y el módulo Bluetooth HC-05 en estado `DESARMADO` (todos los indicadores visuales y audibles apagados).

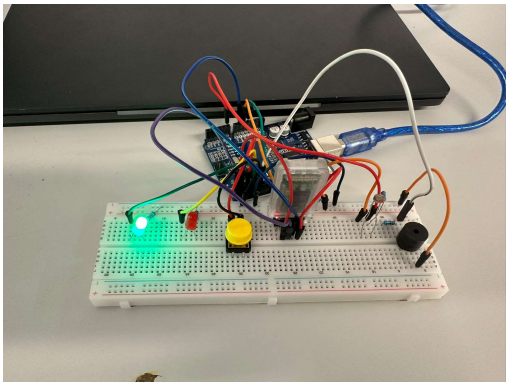


Figura 28. Verificación del sistema en estado `ARMADO`: al recibir la orden inalámbrica, se ilumina el LED verde (`PD3`) y se toma la lectura analógica de referencia $luzBase$ mediante la LDR.

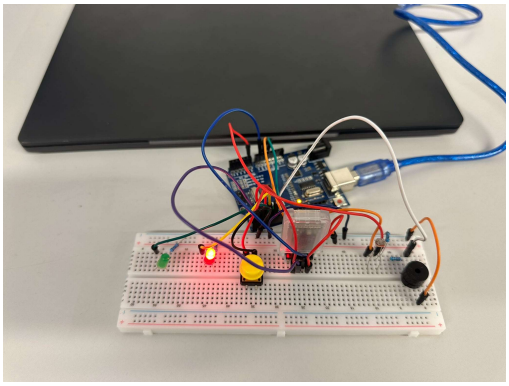


Figura 29. Respuesta del circuito ante la simulación de una intrusión local (bloqueo de luz en LDR o pulsación en `PD2`): se conmuta el indicador visual al LED rojo (`PD4`) y se activa la sirena acústica (`PD5`).

VII-I2. Resultados Operacionales:

1. **Verificación del Enlace Serie:** El periférico USART operando a 9600 bps estableció una comunicación bidireccional limpia y sin pérdida de tramas, enviando la cadena inicial `"SISTEMA:INICIALIZADO"` e interpretando comandos de un solo carácter (`'A'`, `'D'`, `'F'`, `'S'`) al instante.
2. **Prueba de Autocalibración LDR:** Al armar la alarma con distintos niveles de iluminación simulada, el algorit-



Figura 30. Captura de pantalla de la aplicación móvil de Terminal Bluetooth comunicándose a 9600 bps con el microcontrolador, evidenciando el envío de comandos (`ARMAR`, `SILENCIAR`, `MANUAL`, `FORZAR_ALARMA`) y la recepción de reportes de zona en tiempo real.

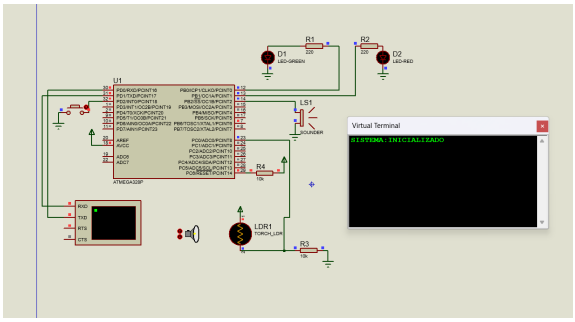


Figura 31. Estado S_0/S_1 (Inicialización y Desarmado): la Virtual Terminal muestra la trama `"SISTEMA:INICIALIZADO"`. Todos los actuadores (`LED_VERDE`, `LED_ROJO` y `SOUNDER`) permanecen en estado lógico bajo (OFF).

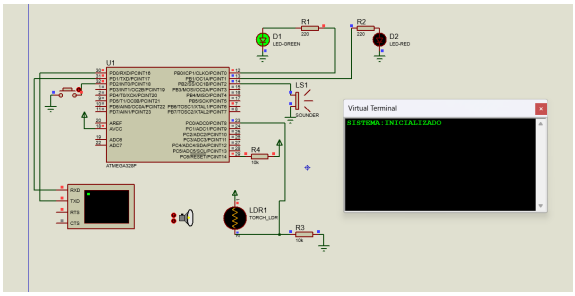


Figura 32. Estado S_2 (Sistema Armado): al ingresar el comando `'A'` en la Virtual Terminal, el microcontrolador conmuta el pin `PB0` a nivel alto, encendiendo el `LED_VERDE` (D1) y guardando la lectura analógica actual del LDR como la referencia $luzBase$.

mo capturó dinámicamente la variable $luzBase$, previniendo disparos erróneos y respondiendo únicamente a desviaciones superiores a $\Delta = \pm 200$ cuentas del ADC.

3. **Evaluación de Modos Operativos:** En `MODO_AUTO`, cualquier evento de intrusión activó secuencialmente los actuadores locales (`LED_ROJO` y `SOUNDER`) y reportó el estado vía serie. En `MODO_MANUAL`, la sirena permaneció silenciada mientras se notificaba la alerta remota.
4. **Interrupción Remota y Silenciado:** La orden `'S'` desactivó inmediatamente la sirena local sin desarmar la FSM, mientras que el comando `'D'` apagó la totalidad de los actuadores y retornó al sistema a su estado de reposo.

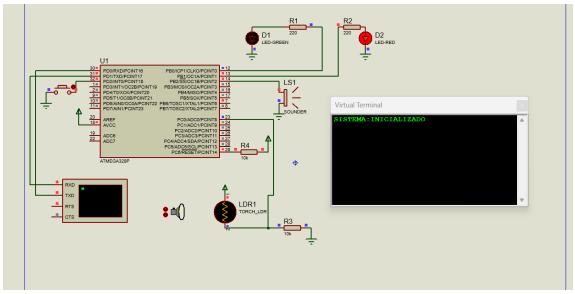


Figura 33. Estado S_3 (Disparo de Alarma): al simular una intrusión (mediante la pulsación del botón en `PD2`, la alteración del nivel de luz en la LDR o el comando `'F'`), el sistema apaga el `LED_VERDE`, enciende el `LED_ROJO` (D2) y activa la sirena acústica `SOUNDER` (LS1).

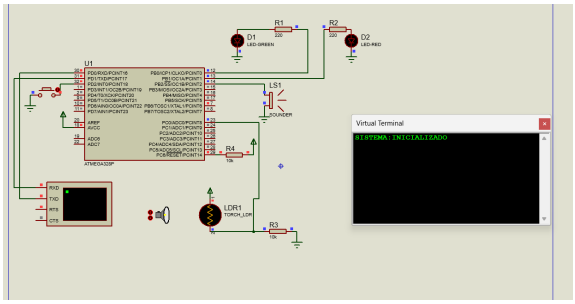


Figura 34. Estado S_4 / Retorno a S_1 (Silenciado y Desarmado): la recepción del comando `'S'` desactiva la alarma sonora manteniendo el testigo visual (`LED_ROJO`), mientras que el comando `'D'` apaga todos los periféricos restableciendo la FSM al estado inicial de reposo.

VII-J. Conclusiones

Se logró diseñar e implementar de manera satisfactoria un sistema de alarma antirrobo supervisado vía Bluetooth en el microcontrolador ATmega328P. La arquitectura basada en Máquina de Estados Finitos garantizó una respuesta predecible y determinista frente a las entradas de los sensores e instrucciones remotas.

Asimismo, la técnica de autocalibración para la fotorresistencia en el conversor analógico-digital demostró ser una solución robusta ante cambios lumínicos del entorno real del laboratorio de UTEC, cumpliendo con todos los requerimientos técnicos y de diseño exigidos.

VIII. Problema D: Juego de Memoria de Secuencias con Joystick, Botón y Matriz LED RGB 8x8 sobre ATmega328P

VIII-A. Objetivo

El objetivo de este ejercicio es desarrollar un juego de memoria utilizando el microcontrolador ATmega328P, una matriz LED RGB WS2812B de 8×8, un joystick analógico y un buzzer pasivo.

El sistema genera una secuencia aleatoria de direcciones que se representa visualmente mediante diferentes figuras y colores en la matriz LED. El jugador debe observar la secuencia y reproducirla utilizando el joystick. Si completa correctamente la secuencia, el juego avanza al siguiente nivel y aumenta la

longitud de la secuencia. En caso de equivocarse, se ejecuta una animación de error y se reinicia la partida.

VIII-B. Metodología

Para desarrollar el sistema se dividió el programa en diferentes funciones, con el objetivo de facilitar su implementación, comprensión y mantenimiento.

En primer lugar, se configuraron los periféricos del ATmega328P necesarios para el funcionamiento del juego. Se utilizó el conversor analógico-digital (ADC) para obtener las posiciones del joystick, entradas digitales para detectar la pulsación de su botón y una salida digital para controlar el buzzer.

Posteriormente, se implementó la comunicación con la matriz WS2812B mediante el envío de datos en serie. Esta comunicación permite controlar individualmente los LED de la matriz y representar las cuatro direcciones del juego mediante figuras triangulares de distintos colores.

A continuación, se desarrolló la lógica principal del juego. El programa genera una secuencia de direcciones, la muestra al jugador y espera que este la reproduzca mediante el joystick. Cada respuesta se compara con el elemento correspondiente de la secuencia. Si todas las respuestas son correctas, se avanza de nivel; si existe un error, se ejecuta la animación correspondiente y se reinicia la partida.

Finalmente, se incorporaron una animación de éxito y el almacenamiento del récord máximo en la EEPROM. También se agregó la comunicación serial para mostrar el valor del récord al iniciar el sistema y cuando se obtiene uno nuevo.

VIII-C. Máquina de Estados Finitos (FSM)

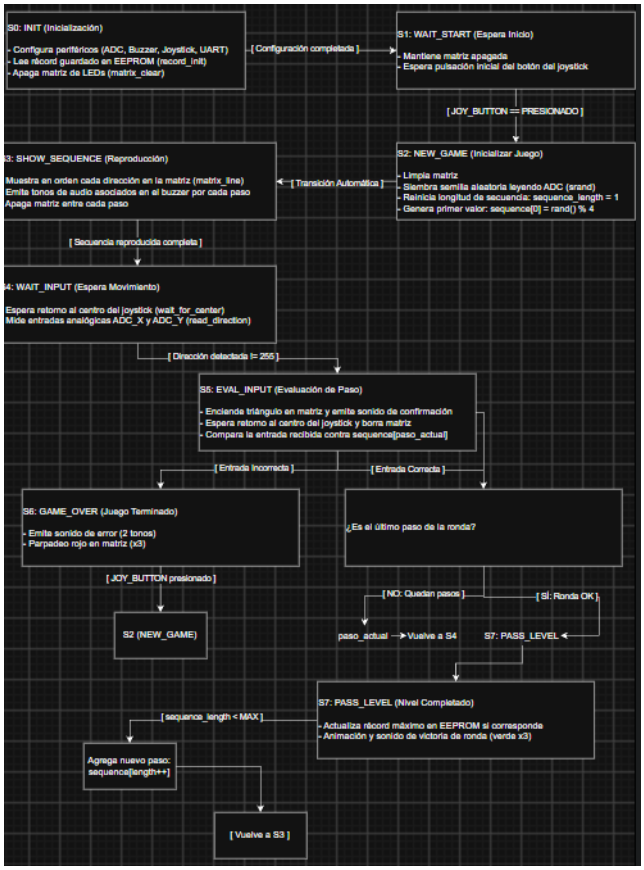


Figura 35. Diagrama de la Máquina de Estados Finitos (FSM) para el Juego de Memoria.

El funcionamiento del juego se organiza mediante una Máquina de Estados Finitos (FSM) que controla el flujo de la partida, desde la inicialización del sistema hasta la comprobación de cada movimiento del jugador. A continuación se describen los estados que conforman el sistema:

- **S0: INIT (Inicialización):** Configura los periféricos del ATmega328P (ADC, Buzzer, Joystick y UART), recupera el récord guardado en la memoria EEPROM (`record_init`) y asegura que la matriz de LED permanezca apagada (`matrix_clear`). Tras completar la configuración, pasa automáticamente a **S1**.
- **S1: WAIT_START (Espera Inicio):** Mantiene la matriz apagada y permanece a la espera de que el usuario presione el botón del joystick (`JOY_BUTTON`) para iniciar la partida.
- **S2: NEW_GAME (Inicializar Juego):** Prepara una nueva partida limpiando la matriz, genera la semilla aleatoria leyendo el valor del ADC (`srand`), reinicia la longitud de la secuencia en 1 (`sequence_length = 1`) y calcula el primer movimiento aleatorio (`sequence[0]`). Pasa automáticamente a **S3**.
- **S3: SHOW_SEQUENCE (Reproducción):** Muestra en orden cada dirección de la secuencia en la matriz (`matrix_line`), emitiendo un tono en el buzzer por cada paso y apagando la matriz brevemente entre cada

elemento. Al terminar de mostrar la secuencia completa, avanza a **S4**.

- **S4: WAIT_INPUT (Espera Movimiento):** Asegura que el joystick haya retornado a la posición central (`wait_for_center`) y lee constantemente los canales analógicos ADC_X y ADC_Y (`read_direction`). Al detectar un movimiento válido del jugador, pasa a **S5**.
- **S5: EVAL_INPUT (Evaluación de Paso):** Muestra la figura correspondiente en la matriz, emite el sonido de confirmación, espera que el joystick vuelva al centro y compara la dirección ingresada contra la almacenada en la secuencia (`sequence[paso_actual]`):
  - *Entrada Incorrecta:* Transiciona al estado de fin de juego **S6**.
  - *Entrada Correcta:* Si quedan pasos por completar en la ronda actual, incrementa el índice y vuelve a **S4**. Si era el último paso de la ronda, avanza a **S7**.
- **S6: GAME_OVER (Juego Terminado):** Reproduce el sonido de error (2 tonos) y hace parpadear la matriz en color rojo 3 veces. Queda a la espera de que el jugador presione el botón del joystick para reiniciar el juego volviendo a **S2**.
- **S7: PASS_LEVEL (Nivel Completado):** Compara el nivel alcanzado con el récord almacenado en la EEPROM y lo actualiza de ser necesario. Ejecuta la animación y sonido de victoria (3 destellos verdes en la matriz). Si no se ha alcanzado la longitud máxima de la secuencia (`MAX`), agrega una nueva dirección (`sequence[length++]`) y regresa a **S3**.

VIII-D. D. Mapeo de Matriz WS2812B y Registro de Récord

La matriz WS2812B está compuesta por 64 LED RGB distribuidos en ocho filas y ocho columnas. Cada LED permite controlar de manera independiente las intensidades de rojo, verde y azul, lo que posibilita representar diferentes colores y figuras.

En este ejercicio se utilizan cuatro figuras triangulares para representar las direcciones del joystick:

- **Arriba:** triángulo verde.
- **Derecha:** triángulo rojo.
- **Abajo:** triángulo azul.
- **Izquierda:** triángulo amarillo.

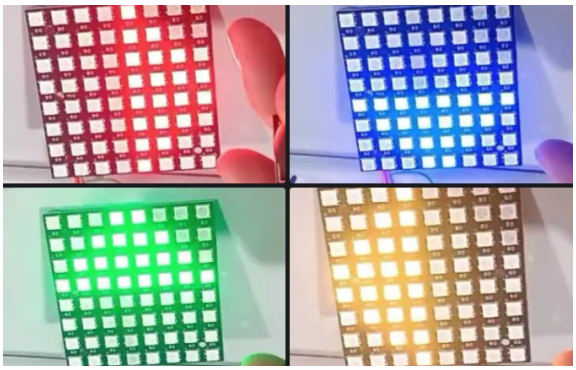


Figura 36. Representación de colores y direcciones en la matriz LED.

La función `matrix_line()` recorre las ocho filas y columnas, determina qué LED debe encenderse según la dirección recibida y envía los valores RGB correspondientes. Los LED que no pertenecen a la figura permanecen apagados.

El programa admite una secuencia de hasta 30 direcciones que establecimos como máximo; sin embargo, esto es modificable si se desea una longitud mayor. Se delimitó a 30 elementos debido a que el ATmega328P cuenta con 2 KB de memoria SRAM y cada posición generada ocupa un byte, reservando memoria para el resto de la pila y variables del programa.

Cada vez que el jugador completa correctamente una secuencia, su longitud aumenta en una unidad, hasta alcanzar el límite definido.

Para conservar el récord se utiliza la memoria EEPROM interna del ATmega328P. Al iniciar el programa, se lee el valor almacenado y se muestra mediante la comunicación serial. Cuando la longitud de una secuencia completada supera el récord anterior, se actualiza el valor en la EEPROM y se muestra el nuevo récord en la consola.

La función `eeprom_update_byte()` permite actualizar el dato solamente cuando su valor cambia, evitando escrituras innecesarias. De esta manera, el récord puede conservarse aunque se apague o reinicie el microcontrolador.

VIII-E. Explicación del Código

El programa está organizado en funciones que separan el control de los periféricos de la lógica del juego.

VIII-E1. Lectura del joystick: Las funciones `adc_init()` y `adc_read()` configuran y utilizan el conversor analógico-digital. Se leen los canales ADC0 y ADC1, correspondientes a las entradas A0 y A1 del Arduino Uno.

VIII-E2. Control de la matriz LED: Las funciones `ws_send_byte()` y `ws_pixel()` permiten enviar los datos a la matriz. La primera transmite cada byte mediante cambios temporizados en la señal digital, mientras que la segunda envía los componentes de color en el orden GRB.

La función `matrix_clear()` apaga todos los LED, mientras que `matrix_line()` dibuja la figura asociada a la dirección recibida.

VIII-E3. Control del buzzer: La función `beep()` genera una señal cuadrada cuya frecuencia y duración se indican como parámetros. A partir de ella se implementan `sound_success()` y `sound_error()`, que producen los sonidos correspondientes a las acciones del juego.

Durante la presentación de la secuencia, cada dirección genera un tono diferente. De esta forma, la información visual se complementa con una señal sonora.

VIII-E4. Generación y verificación de la secuencia: La función `new_game()` reinicia la longitud de la secuencia y genera la primera dirección. La función `show_sequence()` recorre los elementos almacenados y muestra sus figuras en la matriz.

Por su parte, `get_player_direction()` espera una entrada válida del joystick, muestra la dirección detectada y devuelve el valor al programa principal.

En `main()`, las respuestas se comparan con los elementos de la secuencia. Si una respuesta no coincide, se ejecuta `game_over()`. Si todas las respuestas son correctas, se comprueba el récord, se reproduce el sonido de éxito y se ejecuta la nueva animación verde. Finalmente, se agrega una dirección a la secuencia para el siguiente nivel.

VIII-E5. Animación de éxito y memoria EEPROM: La función `success_animation()` enciende todos los LED en verde y los apaga repetidamente para generar tres destellos. Esta animación permite reconocer visualmente que el nivel fue completado correctamente.

Las funciones `record_init()` y `update_record()` se encargan de leer y actualizar el récord máximo. La primera se ejecuta al iniciar el programa y muestra el valor guardado. La segunda compara la longitud de la secuencia completada con el récord anterior y guarda el nuevo valor únicamente cuando se supera.

Las funciones `uart_init()`, `uart_print()` y `uart_print_number()` permiten transmitir el récord mediante la interfaz serial a una velocidad de 9600 baudios.

VIII-F. Montaje Físico y Conexionado

El montaje utiliza un Arduino Uno basado en el microcontrolador ATmega328P, una matriz LED RGB WS2812B de 8×8, un joystick analógico y un buzzer pasivo.

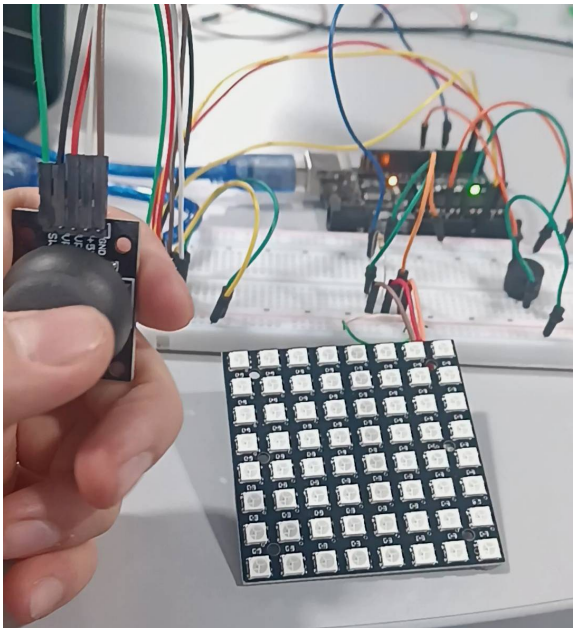


Figura 37. Armado Físico.

Las conexiones utilizadas son las siguientes:

Componente	Terminal	Conexión
Matriz WS2812B	DIN	D6 (PD6)
Matriz WS2812B	VCC	5 V
Matriz WS2812B	GND	GND
Joystick	VRx	A0 (ADC0)
Joystick	VRy	A1 (ADC1)
Joystick	SW	D2 (PD2)
Joystick	VCC	5 V
Joystick	GND	GND
Buzzer pasivo	Positivo	D9 (PB1)
Buzzer pasivo	Negativo	GND

Todos los componentes deben compartir una referencia de tierra común. La matriz se alimenta desde los 5 V del Arduino, de acuerdo con el montaje previsto para el laboratorio. Es importante comprobar que el consumo de corriente se mantenga dentro de los límites seguros de la placa y de la alimentación utilizada.

La comunicación serial utiliza los pines TX y GND para transmitir el récord al equipo. En el Arduino Uno, la transmisión UART corresponde al pin D1 (TX). Para utilizar el monitor serial mediante el USB de la placa no es necesario agregar un cable externo a D1.

VIII-G. Simulación y Resultados

VIII-H. Conclusiones

El desarrollo de este ejercicio permite integrar diferentes periféricos del microcontrolador ATmega328P en un sistema interactivo. La matriz WS2812B se utiliza para representar visualmente las direcciones de la secuencia, mientras que el joystick permite ingresar las respuestas y el buzzer proporciona información sonora durante el juego.

La organización del programa en funciones facilita la comprensión de su funcionamiento y permite separar las tareas de lectura del joystick, control de la matriz, generación de sonidos y verificación de las respuestas.

El uso de la memoria EEPROM agrega la posibilidad de conservar el récord máximo entre reinicios, mientras que la comunicación serial permite consultar su valor sin interferir con la dinámica del juego.

IX. Conclusiones Generales

El desarrollo de las distintas actividades prácticas permitió consolidar el uso del microcontrolador ATmega328P en aplicaciones mecatrónicas de diversa complejidad, logrando una integración eficiente entre periféricos de lectura, actuadores de potencia y módulos de comunicación.

El diseño de firmware estructurado mediante Máquinas de Estados Finitos (FSM) resultó determinante para garantizar un funcionamiento predecible y no bloqueante. Esta arquitectura simplificó el manejo de eventos concurrentes, como la atención simultánea a tramas serie Bluetooth, el escaneo de teclados y

joysticks, y la generación precisa de señales PWM para el control posicional de servomotores y motores DC.

Asimismo, la implementación de buses de comunicación estándar (SPI, TWI/I2C y USART) validó la capacidad del ATmega328P para operar tanto de forma autónoma como en esquemas distribuidos Maestro-Esclavo. La gestión optimizada de la memoria EEPROM interna demostró ser una solución robusta para mantener la persistencia no volátil de parámetros críticos, garantizando la conservación de saldos, claves de seguridad, récords históricos y offsets de calibración frente a interrupciones en la alimentación.

Finalmente, la integración del flujo de trabajo entre simulación en Proteus VSM y montaje físico en protoboards permitió verificar y ajustar el comportamiento del código ante ruido eléctrico, variaciones de luz ambiental y tiempos de respuesta en tiempo real, cumpliendo satisfactoriamente con los objetivos de la unidad curricular.

Referencias

[1] Microchip Technology Inc., *ATmega328P 8-bit AVR Microcontroller Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.com/view.jsp?Searchword=Atmega328p%20datasheet

[2] NXP Semiconductors, *MFRC522 Standard Performance MIFARE and NTAG Frontend Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.com/datasheet-pdf/pdf/227839/NXP/MFRC522.html

[3] Aosong Electronics Co., Ltd., *DHT11 Digital Humidity and Temperature Sensor Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.com/view.jsp?Searchword=Dht11%20datasheet

[4] InvenSense Inc., *MPU-6050 Six-Axis (Gyro + Accelerometer) Product Specification*, AllDataSheet. Disponible en: https://www.alldatasheet.es/datasheet-pdf/pdf/517744/ETC1/MPU-6050.html

[5] Makershop, *I2C LCD Interface Module (PCF8574) Specification and User Manual*. Disponible en: https://www.makershop.de/download/I2C-LCD-interface.pdf

[6] Worldsemi Co., Ltd., *WS2812B Intelligent Control LED Integrated Light Source Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.com/view.jsp?Searchword=WS2812B

[7] Epson, *Stepper Motor EM-91 Specification and Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.es/category/index.jsp?sSearchword=Stepper-Motor%20EM-91

[8] TowerPro, *SG90 Micro Servo Motor Specification and Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.es/datasheet-pdf/pdf/1572383/ETC/SG90.html

[9] Guangzhou HC Information Technology Co., Ltd., *HC-05 Serial Bluetooth Module User Manual & Datasheet*, AllDataSheet. Disponible en: https://www.alldatasheet.es/datasheet-pdf/pdf/1492299/ETC/HC-05.html

X. Enlace al Repositorio y Archivos

- **Carpeta en Google Drive / Entregables:** https://drive.google.com/drive/folders/1CHcE9pehgSDH8hHjwaq2KjZcRxNg2lsk?usp=sharing